# REFLEX

## Delivery Coordination Platform for Small Kenyan Retailers

*Software Requirements Specification (SRS)*

Version 1.0

Prepared as part of the Reflex Readiness Sprint — Professional Learning Program (PLP)

# 1. Introduction

## 1.1 Purpose

This document specifies the functional and non-functional requirements for Reflex, a delivery coordination system for small Kenyan retailers such as electronics shops, pharmacies, and hardware stores. Reflex replaces informal coordination over WhatsApp and phone calls with a structured system that records who is assigned to a delivery, tracks its status in real time, and captures proof of delivery. This SRS is intended to align the development team, panel reviewers, and business stakeholders on exactly what the system must do before and during the build.

## 1.2 Scope

Reflex covers three workflows: logging a delivery request, assigning that request to a rider, and updating and tracking its status through to confirmed delivery. The system is scoped as a one-week sprint build. It is not intended to handle payments, route optimization, or multi-branch retailer accounts — those are noted as roadmap items, not current scope.

## 1.3 Definitions, Acronyms, and Abbreviations

| Term | Definition |
|---|---|
| Retailer | The shop staff member who logs a new delivery request. |
| Dispatcher | The user who reviews open requests and assigns them to a rider. |
| Rider | The person who physically delivers the item and updates its status. |
| Delivery | A single request record tracked from creation to confirmed delivery. |
| Status | The current stage of a delivery: Open, Assigned, Picked Up, or Delivered. |
| Confirmation Code | A short code generated at assignment and required to mark a delivery Delivered. |
| SRS | Software Requirements Specification — this document. |

### 1.4 Stakeholders

| Stakeholder | Interest in the System |
|---|---|
| Retailer staff | Needs a reliable way to log deliveries and see their status without phoning the dispatcher. |
| Dispatcher | Needs visibility into all open requests and a simple way to assign work to riders. |
| Rider | Needs a clear list of assigned deliveries and a simple way to update status. |
| PLP panel / instructors | Assess architecture reasoning, trade-off awareness, and defensibility of decisions. |
| Development team (you) | Uses this SRS as the single reference for what to build and why. |

## 2. Overall Description

### 2.1 Product Perspective

Reflex is a new, standalone web application. It is not a modification of an existing system and has no dependency on external delivery platforms. The system consists of a browser-based client, an Express server, and a SQL database, all newly built for this sprint.

### 2.2 User Classes and Characteristics

| User Class | Technical Skill | Primary Goal |
|---|---|---|
| Retailer staff | Basic — comfortable with simple web forms | Log a delivery quickly and check its status |
| Dispatcher | Basic to moderate | Assign open requests to available riders with minimal delay |
| Rider | Basic — mobile browser use only | Update delivery status accurately while on the move |

### 2.3 Operating Environment

- Client: any modern desktop or mobile browser (Chrome, Safari, Firefox) — no native app required.
- Server: Node.js runtime (v18+), Express framework.
- Database: SQL database (SQLite for development/demo; MySQL-compatible for a production path).
- Network: assumes intermittent to stable internet connectivity typical of small retail environments in Kenya.

### 2.4 Assumptions and Dependencies

- Users access Reflex through a shared or personal device with a browser — no dedicated hardware is required.
- Each user is assigned exactly one role (retailer, dispatcher, or rider) for the duration of the sprint build.

- A working internet or local network connection is available at the point of status updates.
- The system does not need to integrate with existing retailer point-of-sale or inventory systems in this phase.

# 3. Functional Requirements

## 3.1 Retailer Module

| ID | Requirement |
|---|---|
| FR-1.1 | The system shall allow a retailer to create a delivery request with customer name, phone number, address, and item description. |
| FR-1.2 | The system shall validate that all required fields are filled before a request is submitted. |
| FR-1.3 | The system shall allow a retailer to view the current status of any delivery they have created. |
| FR-1.4 | The system shall assign each new delivery request an initial status of 'Open'. |

## 3.2 Dispatcher Module

| ID | Requirement |
|---|---|
| FR-2.1 | The system shall display a list of all delivery requests with status 'Open' to the dispatcher. |
| FR-2.2 | The system shall allow the dispatcher to assign an open request to a specific rider. |
| FR-2.3 | The system shall change a delivery's status to 'Assigned' once a rider is assigned. |
| FR-2.4 | The system shall generate a unique confirmation code at the point of assignment, to be used later at delivery confirmation. |
| FR-2.5 | The system shall record which dispatcher performed each assignment, with a timestamp. |

## 3.3 Rider Module

| ID | Requirement |
|---|---|
| FR-3.1 | The system shall display a list of deliveries currently assigned to the logged-in rider. |
| FR-3.2 | The system shall allow the rider to update a delivery's status to 'Picked Up'. |
| FR-3.3 | The system shall allow the rider to update a delivery's status to 'Delivered' only after a valid confirmation code is entered. |
| FR-3.4 | The system shall reject a 'Delivered' status update if the entered confirmation code does not match the stored code. |

### 3.4 Status Tracking and History

| ID | Requirement |
|---|---|
| FR-4.1 | The system shall record every status change in a status history log, including delivery ID, new status, who changed it, and when. |
| FR-4.2 | The system shall allow any user to view the current status of a delivery by its ID. |
| FR-4.3 | The system shall enforce that status can only progress in the order Open → Assigned → Picked Up → Delivered. |

### 3.5 Synchronization

| ID | Requirement |
|---|---|
| FR-5.1 | The system shall refresh the dispatcher's open-request list automatically at a regular interval, without requiring a manual page reload. |
| FR-5.2 | The system shall refresh a rider's assigned-delivery list automatically at a regular interval. |
| FR-5.3 | The system shall reflect a status change to the retailer's tracking view within a bounded, predictable delay (see NFR-1.1). |

## 4. Non-Functional Requirements

### 4.1 Performance

| ID | Requirement |
|---|---|
| NFR-1.1 | Status updates shall propagate to other views within 5 seconds under the polling-based sync model. |
| NFR-1.2 | The system shall support at least 10 concurrent users without noticeable slowdown, consistent with a single-branch pilot scale. |
| NFR-1.3 | Core pages (request form, dispatcher list, rider list) shall load within 2 seconds on a standard broadband or 4G connection. |

### 4.2 Security

| ID | Requirement |
|---|---|
| NFR-2.1 | Each user shall be assigned a role (retailer, dispatcher, rider), and server-side routes shall check role before allowing an action. |
| NFR-2.2 | Confirmation codes shall not be predictable or sequential. |
| NFR-2.3 | Customer personal data (name, phone, address) shall only be visible to authenticated dispatcher and rider accounts, not exposed publicly. |

### 4.3 Usability

| ID | Requirement |
|---|---|
| NFR-3.1 | Each role shall have a dedicated, single-purpose interface showing only the actions relevant to that role. |
| NFR-3.2 | Forms shall provide clear inline feedback when a required field is missing or invalid. |
| NFR-3.3 | The interface shall be usable on both desktop and mobile browsers without horizontal scrolling. |

### 4.4 Reliability

| ID | Requirement |
|---|---|
| NFR-4.1 | Every status change shall be persisted to the database before being reflected as successful to the user. |
| NFR-4.2 | The status history log shall be immutable — records shall be appended, never overwritten or deleted. |

### 4.5 Compliance and Data Handling

| ID | Requirement |
|---|---|
| NFR-5.1 | Customer contact data shall be retained only for as long as needed to complete and audit a delivery. |
| NFR-5.2 | The system shall not share customer data with any third party or external service. |

## 5. System Architecture Overview

Reflex is built as a single-language stack — JavaScript across client and server, with SQL for persistence — to keep the sprint build simple to implement, explain, and defend.

| Layer | Technology | Responsibility |
|---|---|---|
| Client | HTML, CSS, vanilla JavaScript (one page per role) | Collects input, displays lists, polls server for updates |
| Server | Node.js + Express | Exposes REST routes, applies role checks and status-transition rules |
| Database | SQL (SQLite for demo; MySQL-compatible) | Stores users, deliveries, and status history |

Real-time behavior is approximated through periodic polling rather than persistent WebSocket connections; this trade-off is documented in Section 6.

## 6. Constraints

- Development timeline is fixed at one week (five working days), per the sprint schedule.
- Technology stack is fixed to JavaScript, HTML, CSS, and SQL — no additional frameworks or external real-time services.
- The build must be demonstrable live and defended under direct cross-examination by a review panel.
- No dedicated scanning hardware is available; proof-of-delivery relies on a software-generated confirmation code.

## 7. Assumptions

- User accounts and roles are pre-seeded for the sprint; a full registration/login flow is out of scope.
- A short, tolerable delay (up to 5 seconds) between a status change and its appearance elsewhere is acceptable to end users.
- Riders have access to a smartphone or basic device with browser access at the point of delivery.

## 8. Summary

Reflex gives small Kenyan retailers a structured alternative to WhatsApp-based delivery coordination. Retailers log requests, dispatchers assign them to riders, and riders update status through to a code-verified delivery confirmation — with every change recorded in an auditable history. The system is built on a deliberately simple, single-language stack (JavaScript, HTML, CSS, SQL) suited to a one-week build, using polling rather than persistent connections to approximate real-time sync. This SRS defines 15 functional requirements across five modules and 11 non-functional requirements across performance, security, usability, reliability, and compliance, giving the development team and review panel a shared, unambiguous reference for what Reflex must do and why each decision was made.